So sánh các bộ phân lớp tự cài đặt trên bài toán nhận dạng chữ số viết tay MNIST

Gaussian Naïve Bayes, softmax, SVM tuyến tính, MLP, AdaBoost-SAMME và stacking; CNN là phần nâng cao

---

Trần Trinh Nhân (24280088) Lâm Nhật Tiến (24280027)

Môn Nhận dạng, Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Đồ án giữa kỳ

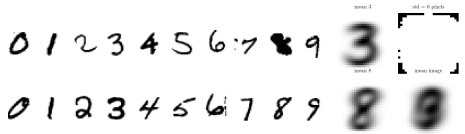
Bài toán và các việc đã làm

Bài toán. MNIST: ảnh xám 28×28, 10 chữ số; 60 000 ảnh huấn luyện, 10 000 ảnh test.

Yêu cầu bắt buộc (đã làm)

- Tiền xử lý và khảo sát dữ liệu.
- Bộ phân lớp tự cài đặt: Naïve Bayes, softmax, SVM, MLP, AdaBoost; ensemble stacking.
- Accuracy, precision, recall, F1 macro, ma trận nhầm lẫn trên train, validation, test.
- Thí nghiệm siêu tham số có biểu đồ.
- So sánh ưu/nhược điểm.

Nâng cao (đã làm): CNN so với MLP, tăng cường dữ liệu, Fashion-MNIST và EMNIST Digits, demo trên web.



Mẫu mỗi chữ số, ảnh trung bình lớp 3 và 8, pixel có phương sai 0 (đen), ảnh trung bình toàn tập.

Dữ liệu, tiền xử lý và phạm vi tự cài đặt

- Chia phân tầng từ 60 000 ảnh: 50 000 train, 10 000 validation (seed 42); test chính thức giữ riêng.
- Làm phẳng ảnh thành vectơ 784 chiều, chuẩn hóa bằng $(\bar{x}, s) = (0.1306, 0.3081)$ tính *chỉ trên train*.
- Siêu tham số chọn trên validation; test chỉ để báo cáo.
- Quét từng siêu tham số một (công cụ tự viết, không dùng Optuna).

Tự viết	**Thư viện**
Phần tính toán lõi, chỉ số, chia tập	scikit-learn, chỉ để đối chiếu
MLP, CNN: lan truyền tiến và ngược, SGD momentum, im2col	PyTorch chỉ làm tensor và phép nhân ma trận; không dùng autograd, lớp dựng sẵn hay bộ tối ưu

Naïve Bayes, hồi quy softmax và SVM tuyến tính

Gaussian Naïve Bayes (các pixel độc lập có điều kiện theo lớp)

$$\hat{y} = \arg\max_k \left[\log \pi_k - \tfrac{1}{2} \sum_{j=1}^{784} \left(\log(2\pi\sigma_{kj}^2) + \frac{(x_j - \mu_{kj})^2}{\sigma_{kj}^2} \right) \right]$$

- Tương quan trung bình giữa hai pixel kề nhau là 0.65: giả thiết độc lập không đúng.
- Validation: 56.05% ở `var_smoothing`=10^{-9}, 79.67% ở 0.1.

Hồi quy softmax, mini-batch gradient descent

$$p_k = \frac{e^{z_k}}{\sum_m e^{z_m}}, \quad z = W^\top x + b, \qquad \nabla_W L = \tfrac{1}{B} X^\top (P - Y) + \lambda W$$

- Accuracy trên chính tập train chỉ đạt 93.62%: bị giới hạn bởi biên tuyến tính.

SVM tuyến tính (hinge one-vs-rest, subgradient)

$$L = \tfrac{1}{B} \sum_{i,k} \max(0, 1 - t_{ik} z_{ik}) + \tfrac{\lambda}{2} \|W\|_F^2, \qquad t_{ik} = \pm 1$$

- Test 91.95% (`LinearSVC`: 92.02%); cùng giới hạn tuyến tính như softmax.

MLP huấn luyện bằng lan truyền ngược viết tay

$$z_l = a_{l-1} W_l + b_l, \quad a_l = \mathrm{ReLU}(z_l) \odot m_l$$

$$\delta_L = \tfrac{1}{B}(\mathrm{softmax}(z_L) - Y)$$

$$\delta_{l-1} = (\delta_l W_l^{\top}) \odot \mathbb{1}[a_{l-1} > 0] \odot m_{l-1}$$

$$\nabla_{W_l} = a_{l-1}^{\top} \delta_l, \quad v \leftarrow \mu v + g, \quad \theta \leftarrow \theta - \eta v$$

- Cấu hình chọn trên validation: hai lớp ẩn $(512, 256)$, $\eta = 0.05$, $\mu = 0.9$, batch 128; 535 818 tham số.
- Khởi tạo He, dropout, dừng sớm (dừng ở epoch 14, khôi phục epoch 9).
- Kiểm thử: gradient khớp sai phân hữu hạn và autograd.

AdaBoost-SAMME và ensemble stacking

$$\alpha_t = \nu\Big(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1)\Big), \quad w_i \leftarrow \frac{w_i\, e^{\alpha_t \mathbb{1}[h_t(x_i)\neq y_i]}}{Z_t}, \quad H(x) = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(x)=k]$$

- Weak learner: cây CART có trọng số, Gini impurity, dựng bằng histogram 32 bin (tự viết bằng NumPy).
- Cấu hình: 393 cây, độ sâu 10, $\nu = 0.1$, mỗi cây xét một nửa số đặc trưng.
- Chi phí: 817.9 s huấn luyện, 18.705 ms mỗi ảnh, tệp xuất 9.2 MiB.
- Điểm số là tỉ lệ phiếu có trọng số, chưa hiệu chỉnh thành xác suất.

Stacking: meta-learner softmax trên $\log p$ (hoặc điểm SVM) của sáu mô hình, 60 đặc trưng

- Meta-learner học trên tập *validation* (hold-out), $\lambda = 0.0001$ chọn bằng kiểm định chéo 5 phần trong validation: 98.69%.
- Test 99.05%, hơn CNN 0.25 điểm; bỏ phiếu mềm không học chỉ 97.92%.

Kết quả trên train, validation, test và đối chiếu scikit-learn

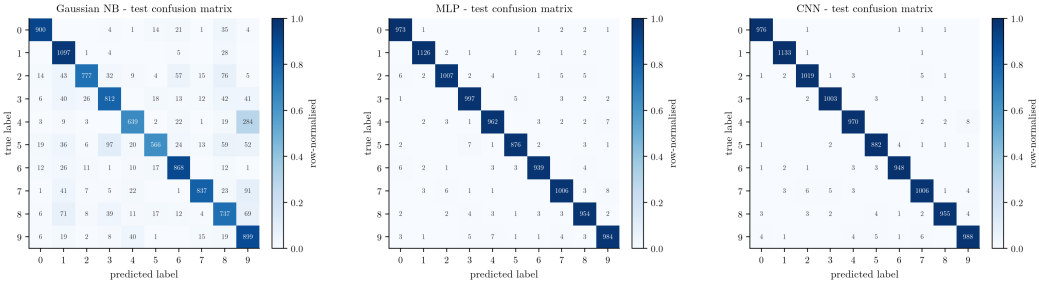
Mô hình	Train	Val	Test	$F1_{macro}$
Gaussian NB	80.13	79.67	81.32	81.16
Softmax	93.62	92.15	92.52	92.42
MLP	99.87	98.18	98.24	98.23
AdaBoost	100.00	97.59	97.60	97.58
CNN	99.40	98.65	98.80	98.79
SVM	92.53	91.38	91.95	91.82
Stacking*	99.83	99.12	99.05	99.04

Accuracy (%), một seed (42). Precision, recall, F1 macro trên cả ba tập: báo cáo, Bảng 2. *Validation là dữ liệu huấn luyện của meta-learner.

Mô hình	Nhóm	sklearn	Chênh
Naïve Bayes	81.32	81.32	0.00
Softmax	92.52	91.86	0.66
MLP	98.24	98.02	0.22
AdaBoost	97.60	97.50	0.10
SVM	91.95	92.02	−0.07

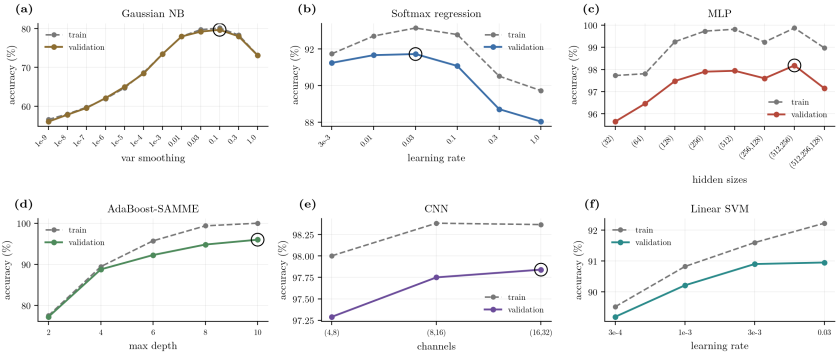
Cùng siêu tham số. Softmax chênh vì L-BFGS của scikit-learn chưa hội tụ.

Ma trận nhầm lẫn trên tập test



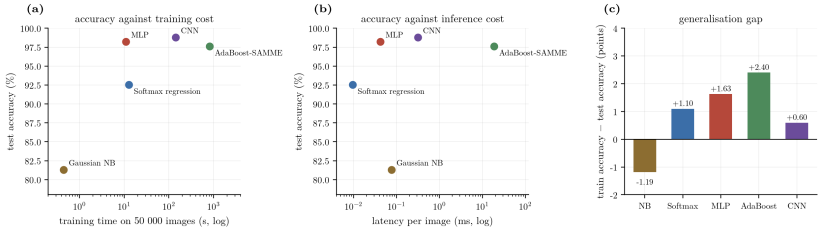
Naïve Bayes, MLP, CNN. Lỗi trên test: Naïve Bayes 1868, softmax 748, SVM 805, MLP 176, AdaBoost 240, CNN 120, stacking 95. Các mô hình nhầm nhiều nhất ở cặp có nét chung: $4 \to 9$, $7 \to 9$, $5 \to 3$.

Ảnh hưởng của siêu tham số



Vòng tròn: giá trị được chọn. AdaBoost (độ sâu 10) và CNN (kênh $(16, 32)$) đạt tốt nhất ở mép khoảng đã quét.

Ưu và nhược điểm của các mô hình



Mô hình	Độ chính xác	Thời gian (huấn luyện; một ảnh)	Khả năng giải thích
Naïve Bayes	thấp nhất, 81.32%	0.4 s; 0.076 ms	μ, σ theo lớp đọc được
Softmax	92.52%, giới hạn bởi biên tuyến tính	12.7 s; 0.010 ms	trọng số vẽ thành ảnh
MLP	98.24%	11.0 s; 0.042 ms	đơn vị ẩn khó đọc
SVM	91.95%, như softmax	11.5 s; 0.008 ms	trọng số vẽ thành ảnh; không có xác suất
AdaBoost	97.60%	817.9 s; 18.705 ms	tần suất pixel; điểm là tỉ lệ phiếu
Stacking	cao nhất, 99.05%	tổng 6 mô hình; 23.8 ms	khó: chỉ biết trọng số từng mô hình

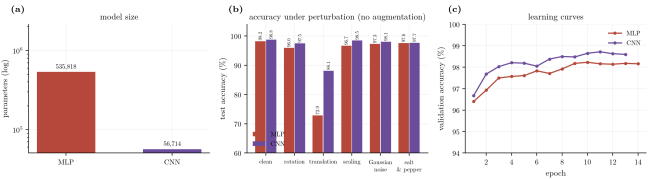
CPU, một máy; thời gian có thể chênh tới gấp đôi giữa các lần chạy.

Nâng cao: CNN so với MLP

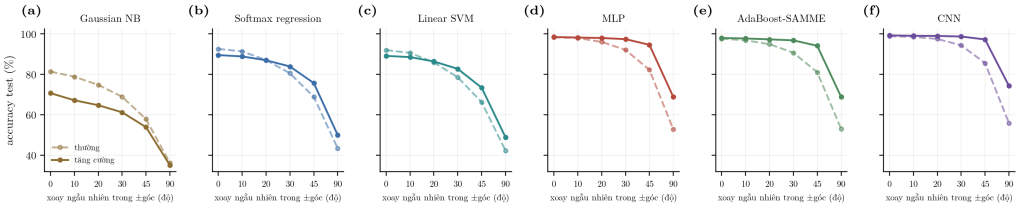
$1{\times}28{\times}28 \to 16{\times}26{\times}26 \to 16{\times}13{\times}13 \to 32{\times}11{\times}11 \to 32{\times}5{\times}5 \to 800 \to 64 \to 10$

56 714 tham số (MLP: 535 818).

- Test: 98.80 so với 98.24%; train − test: 0.60 so với 1.63 điểm.
- Ảnh dịch 3 pixel: 88.14 so với 72.86%.
- Huấn luyện 143.0 s so với 11.0 s; 0.315 ms so với 0.042 ms mỗi ảnh.



Nâng cao: tăng cường dữ liệu



Xoay, dịch, co giãn, nhiễu Gauss, nhiễu muối tiêu; chỉ áp dụng cho tập train. Hình: bản thường (nét đứt) và bản tăng cường (nét liền; xoay $\pm 30°$, dịch ± 2, co giãn $[0.9, 1.1]$, thêm 2 bản sao) trên ảnh test bị xoay. MLP, AdaBoost, CNN tốt hơn trên cả ảnh sạch và ảnh xoay (CNN: 98.80%→99.21%); Naïve Bayes (81.32%→70.64%), softmax (92.52%→89.48%) và SVM (91.95%→89.08%) *kém hơn* trên ảnh sạch.

Nghiên cứu từng phép với CNN (2 seed): cả năm phép cho test 99.25% so với 98.82%. Với MLP, phép dịch nâng accuracy trên ảnh bị dịch từ 72.1 lên 94.5%.

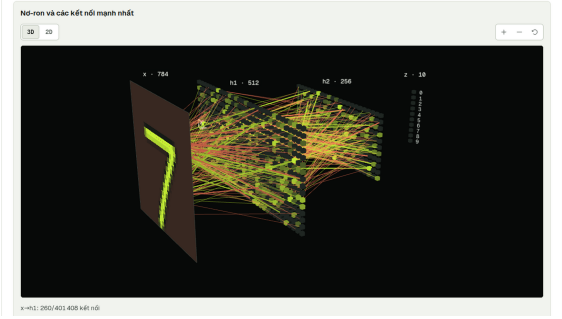
Nâng cao: Fashion-MNIST và EMNIST Digits

Mô hình	MNIST	Fashion-MNIST	EMNIST Digits	MNIST → EMNIST
Gaussian NB	81.32	67.27	80.03	49.80
Softmax regression	92.52	84.27	93.81	53.59
MLP	98.24	88.59	98.36	76.70
AdaBoost-SAMME	97.60	84.74	97.38	73.69
CNN	98.80	88.49	98.64	92.15

Test accuracy (%). Mỗi bộ dữ liệu có phép chia và siêu tham số riêng (quét rút gọn, một seed). Cột cuối: mô hình huấn luyện trên MNIST, đánh giá trên EMNIST Digits (cùng 10 chữ số). Thứ hạng giữ nguyên ở hai bộ chữ số; trên Fashion-MNIST, MLP và CNN chênh 0.10 điểm.

Nâng cao: giao diện demo trên web

- Vẽ hoặc chọn một chữ số, xem dự đoán ngay trong trình duyệt (ONNX Runtime Web); mỗi mô hình hiện cạnh bản tăng cường của nó.
- Mô hình tải từ Hugging Face theo revision cố định.
- Phần “nhìn vào bên trong” vẽ lại phép tính của chính mô hình đã triển khai và đối chiếu với đầu ra ONNX; MLP và CNN vẽ 3D.



Kết luận và phân công

Kết luận

- MNIST: stacking 99.05%, CNN 98.80%, MLP 98.24%, AdaBoost 97.60%, SVM 91.95%, softmax 92.52%, Naïve Bayes 81.32%.
- AdaBoost gần MLP về độ chính xác nhưng đắt hơn nhiều về thời gian.
- Tăng cường dữ liệu chủ yếu cải thiện độ bền với đúng loại biến đổi.

Phân công: Trần Trinh Nhân 50%, Lâm Nhật Tiến 50%. Nhóm có dùng trợ lý AI để hỗ trợ lập trình và điều chỉnh siêu tham số (báo cáo, mục 9).

Mã nguồn: `https://github.com/TranTrinhNhan1/WriteRHand` (kho hiện ở chế độ riêng tư)
Demo: `https://trantrinhnhan1.github.io/WriteRHand/`
Mô hình: `https://huggingface.co/TranTrinhNhan1111/WriteRHand-MNIST`